src/screenshots/c_centralelille_rvb_300sm.png

École Centrale de Lille

Département d'Informatique et de Génie Logiciel

Rapport Académique

Conception et Réalisation d'un Jumeau Numérique pour la Gestion d'un Entrepôt Pharmaceutique

---

Projet : Digital Twin Warehouse

Application Web Interactive de Simulation Logistique 3D

Réalisé par : Mohamed Moustache, Imad Nahya
Bilal Himite, Aude Pinson

Encadrant : Slim Hammadi

Année universitaire : 2025 – 2026

Mars 2026

Table des matières

Table des figures

Liste des tableaux

Chapitre 1

Introduction Générale

1.1 Contexte et problématique

La logistique pharmaceutique constitue l'un des domaines les plus exigeants en matière de gestion d'entrepôt. Les contraintes réglementaires strictes, la nécessité de traçabilité complète des produits, les conditions de stockage spécifiques et les impératifs de rapidité dans la préparation des commandes imposent une organisation irréprochable des flux physiques et informationnels. Dans ce contexte, les gestionnaires d'entrepôts pharmaceutiques font face à des défis majeurs : optimiser les parcours des opérateurs, anticiper les ruptures de stock, équilibrer la charge de travail entre les équipes, et garantir un contrôle qualité rigoureux avant l'expédition des commandes.

Traditionnellement, l'amélioration des processus logistiques repose sur des observations terrain, des études de temps et mouvements, et des modèles théoriques simplifiés. Cependant, ces approches présentent des limites importantes : elles ne permettent pas de tester des scénarios alternatifs sans perturber les opérations réelles, elles peinent à capturer la complexité des interactions entre les différents acteurs de l'entrepôt, et elles offrent une vision statique d'un système intrinsèquement dynamique.

C'est dans ce contexte que le concept de **jumeau numérique** (Digital Twin) prend tout son sens. Un jumeau numérique est une réplique virtuelle fidèle d'un système physique, capable de simuler son comportement en temps réel, d'analyser ses performances et de prédire l'impact de modifications potentielles. Appliqué à la logistique d'entrepôt, il permet de créer un environnement de test sécurisé où les décideurs peuvent expérimenter librement sans risque pour les opérations réelles.

1.2 Objectifs du projet

Le présent projet vise à concevoir et réaliser un jumeau numérique complet pour un entrepôt pharmaceutique. Les objectifs spécifiques sont les suivants :

— Développer un **moteur de simulation** capable de modéliser fidèlement les flux logistiques d'un entrepôt pharmaceutique, incluant la réception des marchandises, le stockage, la préparation des commandes, le contrôle qualité et l'expédition.

— Créer une **visualisation tridimensionnelle** immersive et réaliste de l'entrepôt, permettant de suivre en temps réel les déplacements des agents et l'évolution des stocks.

— Intégrer un **tableau de bord analytique** avec des indicateurs clés de performance (KPI) calculés dynamiquement.

— Implanter un **système d'alertes** pour la détection automatique des anomalies et situations critiques.

— Offrir une fonctionnalité de **scénarios hypothétiques** (What-If) permettant de tester l'impact de différentes configurations sur les performances globales.

— Concevoir une **interface utilisateur** ergonomique et entièrement exécutable dans un navigateur web, sans installation de logiciel spécialisé.

src/screenshots/screenshot_01.png

FIGURE 1.1 – Aperçu global de l'entrepôt 3D simulé en temps réel

1.3 Organisation du rapport

Ce rapport est structuré en plusieurs chapitres qui couvrent l'ensemble des aspects du projet. Le chapitre 2 présente l'état de l'art des jumeaux numériques et de la simulation logistique. Le chapitre 3 décrit l'architecture technique retenue et les choix technologiques effectués. Le chapitre 4 détaille la modélisation de l'entrepôt et sa configuration. Le chapitre 5 expose le moteur de simulation et ses algorithmes. Le chapitre 6 traite de la modélisation 3D et du rendu visuel. Le chapitre 7 présente le tableau de bord et les systèmes d'analyse. Enfin, le chapitre 8 conclut ce rapport et propose des perspectives d'évolution.

Chapitre 2

État de l'Art

2.1 Les jumeaux numériques : définition et évolution

Le concept de jumeau numérique a été formalisé pour la première fois par Michael Grieves en 2002, dans le cadre de la gestion du cycle de vie des produits (Product Lifecycle Management). L'idée fondamentale consiste à créer un modèle virtuel qui évolue en parallèle avec son homologue physique, reflétant en permanence son état actuel et permettant de simuler son comportement futur.

Au fil des années, les jumeaux numériques ont évolué au-delà de la simple réplication statique pour devenir des systèmes dynamiques intégrant des capacités de simulation, d'analyse et de prédiction. Dans le domaine industriel, ils sont désormais utilisés pour la maintenance prédictive, l'optimisation des processus de fabrication, la planification de la chaîne d'approvisionnement et la gestion des infrastructures critiques.

L'application des jumeaux numériques à la logistique d'entrepôt représente une évolution naturelle de cette technologie. Un entrepôt est un système complexe où de multiples agents (opérateurs, machines, systèmes de convoyage) interagissent dans un espace physique contraint, suivant des règles opérationnelles précises. La modélisation fidèle de ces interactions nécessite des algorithmes sophistiqués et une capacité de rendu visuel performante.

2.2 Simulation à événements discrets en logistique

La simulation à événements discrets (Discrete Event Simulation, DES) est la méthode de référence pour modéliser les systèmes logistiques. Elle repose sur le principe que l'état du système change à des instants discrets, correspondant à des événements spécifiques tels que l'arrivée d'une commande, le début d'une opération de picking, ou le passage d'un colis au contrôle qualité.

Des logiciels professionnels tels que FlexSim, AnyLogic et Arena sont largement utilisés

dans l'industrie pour ce type de simulation. Cependant, ces outils présentent plusieurs limitations : coût de licence élevé, nécessité d'installation locale, courbe d'apprentissage importante, et difficulté de partage des résultats avec les parties prenantes non techniques.

Notre approche se distingue en proposant une simulation directement exécutable dans un navigateur web, éliminant ainsi les barrières d'accès tout en offrant une qualité de visualisation comparable aux solutions professionnelles.

2.3 Rendu 3D dans le navigateur avec WebGL

L'évolution de WebGL et de ses surcouches (notamment Three.js) a révolutionné les possibilités de rendu 3D dans le navigateur. WebGL fournit un accès direct au GPU depuis le navigateur, permettant un rendu matériellement accéléré avec des performances proches des applications natives.

Three.js, bibliothèque construite au-dessus de WebGL, simplifie considérablement le développement d'applications 3D en fournissant des abstractions de haut niveau pour les scènes, caméras, lumières, matériaux et géométries. Combiné avec React Three Fiber (R3F), qui permet de déclarer des scènes 3D sous forme de composants React, il devient possible de construire des applications complexes mêlant interface utilisateur 2D et visualisation 3D dans une architecture cohérente et maintenable.

2.4 Algorithmes de navigation et de pathfinding

La navigation des agents dans un environnement virtuel est un problème classique en informatique. L'algorithme A* (A-Star), proposé par Peter Hart, Nils Nilsson et Bertram Raphael en 1968, reste la référence pour la recherche de chemin optimal dans un graphe. Sa combinaison d'un coût réel accumulé et d'une heuristique admissible garantit l'optimalité du chemin trouvé tout en limitant l'espace de recherche exploré.

Dans le contexte d'un entrepôt, le graphe de navigation doit refléter fidèlement la topologie des allées et des intersections, interdisant aux agents de traverser les rayonnages ou les murs. La modélisation sous forme de nœuds (intersections) et d'arêtes (segments d'allée) permet de capturer cette contrainte de manière élégante.

Chapitre 3

Architecture Technique et Choix Technologiques

3.1 Vue d'ensemble de l'architecture

L'architecture du système repose sur une séparation stricte entre trois couches fonctionnelles : la couche de **simulation** qui implémente la logique métier pure, la couche de **gestion d'état** qui centralise les données partagées, et la couche de **présentation** qui assure le rendu visuel et l'interaction utilisateur.

Cette séparation garantit que le moteur de simulation peut fonctionner indépendamment du rendu graphique, facilitant les tests unitaires et permettant potentiellement une exécution côté serveur pour des simulations accélérées. Le flux de données suit un cycle unidirectionnel : la configuration alimente le moteur de simulation, dont les résultats sont synchronisés vers un magasin d'état centralisé, lequel alimente à son tour les composants de rendu 3D, le tableau de bord et la liste des commandes.

3.2 Pile technologique

Le choix des technologies a été guidé par plusieurs critères : performances de rendu, réactivité de l'interface, facilité de maintenance, et accessibilité depuis un navigateur web standard.

Table 3.1 – Technologies utilisées dans le projet

Technologie	**Rôle**	**Justification**
React 19	Framework UI	Composants déclaratifs, hooks, gestion du cycle de vie
Vite	Bundler / Serveur de développement	Hot Module Replacement instantané, build optimisé
React Three Fiber	Rendu 3D déclaratif	Scène 3D sous forme de composants React
Three.js	Moteur WebGL	Rendu matériellement accéléré, PBR, ombres
Zustand	Gestion d'état	Store minimaliste sans boilerplate, sélecteurs fins
Tailwind CSS	Stylisation	Classes utilitaires, responsive, thème cohérent
Recharts	Graphiques	Visualisation de données en temps réel
Framer Motion	Animations UI	Transitions fluides pour l'interface 2D

3.3 Architecture des fichiers

Le projet est organisé selon une structure modulaire claire qui reflète la séparation des responsabilités. Le répertoire `simulation` contient la logique métier pure, composée du moteur de simulation et du hook de synchronisation. Le répertoire `store` héberge le magasin d'état Zustand qui centralise toutes les données partagées entre les composants. Le répertoire `components` regroupe l'ensemble des composants React responsables de l'affichage : la carte 3D de l'entrepôt, le tableau de bord des indicateurs, la barre latérale de contrôle, la liste des commandes, le panneau d'alertes, le panneau de scénarios, le modal de configuration des arrivées, et le modal de gestion des quarts de travail.

Le fichier de configuration de l'entrepôt, au format JSON, est stocké dans le répertoire public et chargé dynamiquement au démarrage de l'application. Cette approche permet de modifier la disposition de l'entrepôt sans recompiler l'application.

3.4 Cycle de données unidirectionnel

Le flux de données dans l'application suit un cycle unidirectionnel rigoureux. La configuration JSON alimente le moteur de simulation. À chaque image d'animation, le hook

de synchronisation charge l'état actuel des agents depuis le magasin Zustand, transmet ces données au moteur, récupère les résultats du calcul (positions mises à jour, métriques, alertes, commandes), et synchronise ces résultats vers le magasin. Les composants React, abonnés aux tranches spécifiques du magasin, se mettent à jour automatiquement lorsque les données qu'ils observent changent. Cette architecture élimine les problèmes de synchronisation et garantit une cohérence parfaite entre la simulation et l'affichage.

src/screenshots/screenshot_11.png

FIGURE 3.1 – Interface de suivi en temps réel des commandes et de leur distribution

L'application déploie également une interface permettant d'effectuer des tests "What-If" en manipulant les paramètres à la volée :

src/screenshots/screenshot_10.png

FIGURE 3.2 – Panneau des Scénarios "What-If" avec comparaison de Baseline

Chapitre 4

Modélisation de l'Entrepôt

4.1 Dimensions et disposition générale

L'entrepôt est modélisé sur une grille bidimensionnelle de 1200 par 900 unités. Cette grille définit l'espace de travail dans lequel évoluent les agents et sont positionnées les différentes zones fonctionnelles. La conversion vers l'espace tridimensionnel s'effectue via un facteur d'échelle de 0.05, ce qui transforme l'entrepôt en un espace 3D de 60 par 45 unités Three.js, garantissant des proportions visuelles réalistes dans le navigateur.

4.2 Types de zones fonctionnelles

L'entrepôt est divisé en plusieurs types de zones, chacune jouant un rôle spécifique dans la chaîne logistique.

Table 4.1 – Types de zones de l'entrepôt

Type de zone	**Quantité**	**Rôle**
Picking	24	Zones de prélèvement où les préparateurs collectent les articles
Storage (Réserve)	4	Zones de stockage de réserve pour le réapprovisionnement
Inbound (Réception)	1	Zone de réception des marchandises entrantes
Conveyor (Tapis roulant)	1	Système de convoyage pour le contrôle qualité
Outbound (Expédition)	1	Baie d'expédition des commandes terminées
Workstation	1	Zone de contrôle pilulier

Chaque zone est caractérisée par un identifiant unique, un nom descriptif, des coordonnées de position, des dimensions, un niveau de stock actuel, une capacité maximale et un seuil critique de réapprovisionnement. Les 24 zones de picking sont réparties en groupes

identifiés par des lettres (A à D, E, F1 à F4, G à Q, R1 à R2), reflétant l'organisation physique typique d'un entrepôt pharmaceutique réel.

4.3 Graphe de navigation

Le déplacement des agents dans l'entrepôt ne s'effectue pas en ligne droite. Il suit un graphe de navigation composé de nœuds et d'arêtes qui modélise fidèlement le réseau d'allées de l'entrepôt. Ce graphe comprend 47 nœuds représentant les intersections et les extrémités d'allées, reliés par plus de 50 arêtes bidirectionnelles.

La topologie du graphe reproduit les corridors réels de l'entrepôt : un corridor supérieur traversant horizontalement la partie haute de l'entrepôt, des allées verticales desservant les zones A à E, une allée centrale (allée F) formant la colonne vertébrale de l'entrepôt, des rangées horizontales desservant les zones G à Q, une allée droite pour les zones R1 et R2, et un hub inférieur où convergent les flux principaux.

Les nœuds sont stratégiquement positionnés dans les allées, entre les rayonnages, garantissant qu'aucun agent ne traverse jamais un rack ou un mur lors de ses déplacements. Cette modélisation assure un réalisme visuel essentiel pour la crédibilité de la simulation.

Chapitre 5

Moteur de Simulation

5.1 Architecture du moteur

Le moteur de simulation constitue le cœur algorithmique de l'application. Il est conçu comme un module autonome, totalement indépendant du framework de rendu, et implémente l'ensemble de la logique métier de l'entrepôt : génération des arrivées, création des commandes, gestion des agents, calcul des métriques et détection des alertes.

Le moteur fonctionne selon le principe du **delta time** : le temps réel écoulé entre deux images d'animation est multiplié par un facteur de vitesse configurable par l'utilisateur (1x, 2x, 5x, 10x). Cette approche garantit que le comportement de la simulation est indépendant du taux de rafraîchissement de l'écran. Un ordinateur affichant 30 images par seconde et un autre en affichant 120 produisent exactement les mêmes résultats de simulation sur une durée identique.

L'horloge interne du moteur démarre à 6 heures du matin, simulant le début d'une journée de travail typique dans un entrepôt pharmaceutique. À chaque cycle de mise à jour, le moteur exécute séquentiellement les opérations suivantes : génération des arrivées entrantes par zone, génération des commandes sortantes, animation du convoyeur, mise à jour des agents via leur machine à états, calcul des métriques avancées, et détection des alertes.

5.2 Distribution bimodale des arrivées

L'intensité des arrivées de marchandises dans l'entrepôt suit une distribution bimodale gaussienne, modélisant fidèlement les deux pics d'activité observés dans un entrepôt réel : un pic matinal aux alentours de 9 heures et un pic en début d'après-midi vers 14 heures.

L'intensité est calculée selon la formule :

$$\lambda(h) = (w_1 \cdot G_1(h) + w_2 \cdot G_2(h)) \times \text{maxRate} \tag{5.1}$$

où chaque composante gaussienne est définie par :

$$G_i(h) = \exp\left(-\frac{1}{2}\left(\frac{h - \text{peak}_i}{\sigma_i}\right)^2\right) \tag{5.2}$$

Les paramètres de cette distribution sont configurables indépendamment pour chaque zone de réception : position temporelle de chaque pic, écart-type contrôlant la largeur du pic, poids relatif de chaque composante, et débit maximal en unités par heure. Un mécanisme d'accumulateur fractionnaire convertit le taux continu en événements discrets, assurant une génération réaliste des arrivées. Les marchandises générées sont ajoutées à une file d'attente de réception que les magasiniers consomment progressivement.

src/screenshots/screenshot_05.png

FIGURE 5.1 – Interface de configuration des arrivées bimodales

5.3 Génération des commandes sortantes

Les commandes sortantes suivent également une distribution bimodale, avec un premier pic vers 10 heures et un second vers 15 heures. Lorsqu'une commande est déclenchée, le système vérifie d'abord qu'un plafond de 5 commandes simultanées en attente n'est pas

atteint, sélectionne aléatoirement une zone de picking cible, et détermine une quantité comprise entre 30 et 100 unités.

Chaque commande possède un identifiant unique et suit un cycle de vie complet en cinq états : en attente (*pending*), assignée à un préparateur (*assigned*), déposée sur le convoyeur (*on_ conveyor*), contrôlée (*controlled*) et terminée (*completed*). Chaque transition est horodatée précisément afin de calculer les indicateurs de performance temporels.

5.4 Agents et machines à états finis

L'entrepôt est peupé de sept agents répartis en trois types fonctionnels, chacun gouverné par une machine à états finis (Finite State Machine, FSM) qui détermine son comportement dynamique.

5.4.1 Magasiniers (Storekeepers)

Les trois magasiniers gèrent le flux entrant : réception des marchandises et rangement dans les zones de réserve. Leur première priorité est de traiter les arrivées en attente dans la file de réception. Lorsqu'un arrivage est disponible, le magasinier se déplace vers la zone de réception, récupère la marchandise, puis se dirige vers la zone de réserve cible pour y déposer le stock. En l'absence d'arrivage, le magasinier peut effectuer une tournée d'inspection aléatoire vers une zone de réserve.

5.4.2 Préparateurs (Pickers)

Les trois préparateurs constituent le maillon central de la chaîne logistique. Leur première priorité est la préparation des commandes : le préparateur prend la première commande en attente, se rend dans la zone de picking concernée, et effectue le prélèvement. Le temps de prélèvement est proportionnel à la quantité commandée. Une fois le picking terminé, le préparateur transporte les articles jusqu'au convoyeur.

Leur seconde priorité est le réapprovisionnement. Lorsqu'aucune commande n'est en attente, les préparateurs scannent toutes les zones de picking à la recherche de celles dont le stock est descendu sous le seuil critique. Le réapprovisionnement suit une logique en cascade : d'abord la réserve affiliée, puis toute zone de stockage disponible. Un mécanisme anti-duplication empêche plusieurs agents de réapprovisionner simultanément la même zone.

5.4.3 Contrôleur (Controller)

Le contrôleur unique gère le contrôle qualité et l'expédition. Quand un colis atteint l'extrémité du convoyeur, le contrôleur s'y rend, inspecte le colis pendant une durée déter-

minée, puis le transporte vers la zone d'expédition où la commande est marquée comme terminée.

5.5 Algorithme de pathfinding A*

Tous les déplacements des agents utilisent l'algorithme A* pour déterminer le chemin le plus court dans le graphe de navigation. L'heuristique retenue est la distance euclidienne entre le nœud courant et le nœud de destination, garantissant l'admissibilité de l'heuristique et donc l'optimalité du chemin trouvé.

Le processus de navigation se déroule en trois étapes. D'abord, le système identifie le nœud du graphe le plus proche de la position actuelle de l'agent et le nœud le plus proche de la zone de destination. Ensuite, l'algorithme A* calcule la séquence optimale de nœuds à traverser. Enfin, l'agent se déplace en interpolation linéaire vers chaque nœud successif à une vitesse de 150 unités par seconde, calibrée pour un mouvement visuellement réaliste.

5.6 Système d'évitement de collisions

Un système sophistiqué d'évitement de collisions empêche les agents de se superposer visuellement. À chaque itération, le moteur calcule la distance euclidienne entre chaque paire d'agents. Lorsque cette distance passe sous un seuil de 75 unités, le système active l'évitement.

Le type de rencontre est déterminé par le produit scalaire des vecteurs de direction normalisés des deux agents. Lors d'une rencontre frontale, les deux agents esquivent simultanément vers leur droite respective grâce à un vecteur perpendiculaire, avec une force proportionnelle à la proximité. Lorsqu'un agent en suit un autre, il décélère progressivement. Sous 20 unités, une force de répulsion radiale évite tout chevauchement. L'évitement est désactivé lorsqu'un agent atteint sa destination pour prévenir les phénomènes d'orbite.

Un mécanisme complémentaire de stationnement intelligent garantit que les agents inactifs maintiennent une distance minimale entre eux, se relocalisant automatiquement vers un nœud voisin libre si nécessaire.

5.7 Gestion des quarts de travail

Les magasiniers et les préparateurs suivent des horaires de travail configurables, organisés en deux blocs par jour avec une pause entre les deux. Le système prend en charge les quarts de jour standard, les quarts de soirée et les quarts de nuit avec franchissement de minuit.

À la fin de son quart, un agent ne disparaît pas immédiatement : il termine d'abord sa tâche en cours, puis se dirige vers la sortie de l'entrepôt avant de passer en mode inactif.

Il réapparaît automatiquement au début de son prochain bloc de travail. Le contrôleur, quant à lui, assure une couverture continue 24 heures sur 24.

src/screenshots/screenshot_06.png

FIGURE 5.2 – Panneau de configuration des plannings et quarts de travail

Chapitre 6

Modélisation 3D et Rendu Visuel

6.1 Environnement et matériaux physiques

Le rendu 3D de l'entrepôt exploite les capacités de rendu physiquement réaliste (Physically Based Rendering, PBR) de Three.js. Le sol est texturé avec un matériau simulant le béton, recevant dynamiquement les ombres projetées par l'ensemble des éléments de la scène. L'éclairage combine une lumière directionnelle principale générant des ombres portées nettes, avec une carte d'ombres de haute résolution, et une lumière hémisphérique simulant l'éclairage ambiant typique des grands hangars industriels.

6.2 Conversion des coordonnées 2D vers 3D

Les coordonnées bidimensionnelles du moteur de simulation sont converties en coordonnées tridimensionnelles via un facteur d'échelle de 0.05. L'entrepôt de 1200 par 900 unités en 2D devient un espace de 60 par 45 unités Three.js, centré à l'origine du repère 3D pour faciliter le positionnement de la caméra et l'éclairage.

6.3 Rayonnages et stockage

Les rayonnages sont construits à partir de géométries partagées pour optimiser les performances. Chaque rack est composé de poteaux verticaux cylindriques en métal gris foncé, d'étagères horizontales en gris clair métallique, et de boîtes de stock générées procéduralement en fonction du niveau de stock actuel de la zone. La hauteur des racks varie selon le type de zone, et le nombre d'étagères est calculé dynamiquement, offrant un retour visuel immédiat sur l'état des stocks.

6.4 Animations humanoïdes procédurales

Les agents sont représentés par des personnages humanoïdes animés de manière procédurale, sans recours à des squelettes importés ni à des fichiers d'animation précalculés. L'animation de marche est générée en temps réel : les jambes se balancent en opposition de phase selon une fonction sinusoïdale, le buste oscille verticalement pour imiter les pas, et une transition fluide par interpolation linéaire ramène les membres en position droite lorsque l'agent s'arrête.

6.5 Éléments architecturaux

L'entrepôt est complété par des éléments architecturaux renforçant l'immersion : murs semi-transparents délimitant l'espace, colonnes structurelles en acier, conduits de ventilation au plafond, et fermes triangulaires (trusses) soutenant la toiture. Chaque élément utilise des matériaux PBR distincts avec des propriétés de métallité, de rugosité et de transparence soigneusement calibrées.

src/screenshots/screenshot_02.png

FIGURE 6.1 – Vue de dessus (*Top-Down*) de la disposition de l'entrepôt

src/screenshots/screenshot_03.png

FIGURE 6.2 – Vue détaillée des agents réalisant des opérations de picking

6.6 Convoyeur et colis

Le convoyeur est rendu avec des rouleaux cylindriques en rotation continue et un matériau métallique réaliste. Les colis déposés par les préparateurs sont représentés par des boîtes colorées aléatoirement, positionnées par interpolation linéaire le long du tapis selon leur attribut de progression. Un colis au début du tapis a une progression de 0, tandis qu'un colis arrivé en bout de chaîne a une progression de 1, prêt pour le contrôle qualité.

src/screenshots/screenshot_04.png

FIGURE 6.3 – Vue détaillée du convoyeur central de contrôle qualité

6.7 Optimisations de rendu

Plusieurs optimisations garantissent la fluidité du rendu malgré la complexité de la scène. Les géométries et matériaux sont partagés entre les instances similaires, évitant la duplication en mémoire. Tous les composants 3D sont enveloppés dans des mémorisations React pour éviter les recalculs inutiles. Les textures et calculs coûteux sont mis en cache via des hooks spécialisés. L'interface textuelle 2D est rafraîchie à une fréquence réduite pour libérer des ressources processeur au profit du rendu 3D.

Chapitre 7

Tableau de Bord et Systèmes d'Analyse

7.1 Indicateurs clés de performance

Le tableau de bord affiche en temps réel sept indicateurs clés de performance calculés dynamiquement par le moteur de simulation.

TABLE 7.1 – Indicateurs clés de performance du tableau de bord

Indicateur	**Description**
Temps moyen de préparation	Moyenne des 20 derniers temps écoulés entre l'assignation d'une commande et son dépôt sur le convoyeur
Temps moyen d'attente au contrôle	Moyenne des 20 derniers temps d'attente des colis sur le convoyeur avant le contrôle qualité
Taux d'utilisation des préparateurs	Pourcentage de préparateurs actuellement occupés par rapport au nombre total
Taux de saturation du convoyeur	Pourcentage de la capacité du convoyeur actuellement utilisée
Taux de rupture de stock	Pourcentage de zones de picking dont le stock est épuisé
Commandes en attente	Nombre de commandes ayant le statut en attente
Débit horaire	Nombre de commandes terminées divisé par le nombre d'heures écoulées

Les métriques temporelles sont calculées sur une fenêtre glissante des vingt dernières commandes, évitant que les données initiales ne faussent les indicateurs courants. Cette approche fournit une vision représentative des performances actuelles de l'entrepôt.

src/screenshots/screenshot_07.png

src/screenshots/screenshot_08.png

FIGURE 7.1 – Tableau de bord : indicateurs clés de performance (KPIs)

7.2 Système d'alertes

Le moteur de simulation intègre un système de détection automatique des anomalies qui identifie cinq types de situations critiques : rupture de stock complète (sévérité cri-

tique), stock critique sous la moitié du seuil (avertissement), saturation du convoyeur avec plus de cinq colis en file (avertissement), surcharge des préparateurs lorsque tous sont occupés avec plus de trois commandes en attente (critique), et saturation du contrôle avec plus de trois colis en attente d'inspection (avertissement).

Chaque alerte est caractérisée par un identifiant unique, un type, un niveau de sévérité, un message descriptif et la référence de la zone concernée. Les alertes sont synchronisées vers le magasin d'état centralisé et affichées dans un panneau dédié de l'interface, avec un codage couleur distinguant les avertissements des situations critiques.

src/screenshots/screenshot_09.png

FIGURE 7.2 – Panneau des alertes notifiant les ruptures de stock critiques

7.3 Système de scénarios hypothétiques

Le système de scénarios *What-If* constitue l'un des outils les plus puissants du jumeau numérique. Il permet aux décideurs de tester l'impact de modifications hypothétiques sur les performances de l'entrepôt, sans risque pour les opérations réelles.

Le fonctionnement repose sur un mécanisme de sauvegarde et de restauration. Avant d'appliquer un scénario, le système capture un instantané complet de l'état courant (métriques, paramètres, nombre d'agents). L'utilisateur peut ensuite appliquer l'un des scénarios prédéfinis et observer son impact sur les indicateurs. À tout moment, il peut réinitialiser l'état pour revenir à la configuration de référence.

TABLE 7.2 – Scénarios What-If disponibles

Scénario	**Effet**
Augmentation de 20% des commandes	Augmente le débit maximal de toutes les arrivées et commandes de 20%
Retrait d'un préparateur	Retire le dernier préparateur de la simulation
Pic de commandes frigo	Double le débit des commandes sortantes
Ruptures fréquentes	Force le stock de toutes les zones de picking juste sous le seuil critique
Contrôle ralenti	Double le temps de contrôle qualité

7.4 Horloge de simulation et temporalité

L'interface affiche en permanence l'heure et le jour simulatés, permettant à l'utilisateur de situer les événements dans le temps. La vitesse de simulation est ajustable en temps réel, offrant la possibilité d'accélérer la simulation pour observer les tendances à long terme ou de la ralentir pour analyser un événement spécifique en détail.

7.5 Interface de contrôle

La barre latérale de l'application centralise les contrôles de la simulation : bouton de lecture et de pause, sélection de la vitesse de simulation, accès aux panneaux de configuration des arrivées, des quarts de travail et des scénarios. L'interface est conçue pour être intuitive, permettant même à un utilisateur non technique de maîtriser rapidement l'ensemble des fonctionnalités.

Chapitre 8

Conclusion et Perspectives

8.1 Synthèse des réalisations

Le présent projet a permis de concevoir et de réaliser un jumeau numérique complet pour un entrepôt pharmaceutique, directement exécutable dans un navigateur web. Le système intègre un moteur de simulation reproduisant fidèlement les flux logistiques (réception, stockage, préparation, contrôle qualité, expédition), une visualisation tridimensionnelle immersive avec des animations procédurales réalistes, un tableau de bord analytique avec sept indicateurs clés de performance, un système d'alertes pour la détection d'anomalies, et un mécanisme de scénarios hypothétiques pour l'aide à la décision.

L'architecture technique retenue, fondée sur React, Three.js et Zustand, démontre qu'il est possible de construire des applications de simulation complexes entièrement côté client, sans serveur dédié, tout en maintenant des performances de rendu fluides. La séparation stricte entre logique de simulation et rendu visuel garantit la maintenabilité et l'évolutivité du système.

Les algorithmes mis en œuvre, notamment le pathfinding A*, les machines à états finis pour le comportement des agents, les distributions bimodales pour la génération d'événements, et le système d'évitement de collisions, confèrent à la simulation un réalisme et une fidélité remarquables par rapport aux opérations d'un entrepôt réel.

8.2 Apports du projet

Ce projet apporte plusieurs contributions significatives. Sur le plan technique, il démontre la faisabilité d'un jumeau numérique entièrement web, accessible sans installation. Sur le plan fonctionnel, il offre aux gestionnaires d'entrepôts un outil de simulation et d'analyse complet. Sur le plan pédagogique, il illustre l'intégration de concepts avancés en informatique : algorithmique de graphes, simulation stochastique, rendu 3D temps réel et architecture logicielle modulaire.

8.3 Limites identifiées

Malgré ses atouts, le système présente certaines limites. La simulation s'exécute entièrement côté client, ce qui limite la complexité des scénarios simulables sur des machines peu puissantes. Le graphe de navigation est statique et ne s'adapte pas dynamiquement aux modifications de l'agencement. Les distributions d'arrivées, bien que configurables, restent des modèles simplifiés de la réalité.

8.4 Perspectives d'évolution

Plusieurs pistes d'évolution sont envisagées pour enrichir le système. L'intégration de données réelles en provenance d'un système de gestion d'entrepôt (WMS) permettrait de transformer le jumeau numérique en un outil de supervision en temps réel. L'ajout d'algorithmes d'intelligence artificielle, tels que l'apprentissage par renforcement, pourrait optimiser automatiquement les stratégies d'allocation des agents et de réapprovisionnement. L'extension du modèle à plusieurs étages ou à un réseau d'entrepôts interconnectés élargirait le champ d'application. Enfin, l'intégration de la réalité virtuelle (VR) ou augmentée (AR) offrirait une expérience d'immersion encore plus complète pour la formation des opérateurs.

Bibliographie

[1] Grieves, M. (2002). *Digital Twin : Manufacturing Excellence through Virtual Factory Replication*. White Paper.

[2] Meta Platforms. (2024). *React – A JavaScript library for building user interfaces*. https://react.dev

[3] Three.js contributors. (2024). *Three.js – JavaScript 3D Library*. https://threejs.org

[4] Poimandres. (2024). *React Three Fiber*. https://docs.pmnd.rs/react-three-fiber

[5] Poimandres. (2024). *Zustand – Bear necessities for state management in React*. https://zustand-demo.pmnd.rs

[6] Evan You. (2024). *Vite – Next Generation Frontend Tooling*. https://vitejs.dev

[7] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107.

[8] Wagner, F. (2006). *Modeling Software with Finite State Machines*. Auerbach Publications.

[9] Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). *Discrete-Event System Simulation*. Prentice Hall.

[10] FlexSim Software Products. (2024). *FlexSim Simulation Software*. https://www.flexsim.com

[11] Khronos Group. (2024). *WebGL Specification*. https://www.khronos.org/webgl

[12] Pharr, M., Jakob, W., & Humphreys, G. (2016). *Physically Based Rendering : From Theory to Implementation*. Morgan Kaufmann.